- The *–A* option prints the packet in ASCII format. To print output in both the ASCII and HEX format, use the *–XX* parameter.

## *Tcpdump* scenarios

That's enough of the theory—now for some practical examples. The following example captures two packets of network traffic from TCP port 110 in ASCII format:

```
$ sudo tcpdump -c 2 -A port 110
listening on eth0, link-type EN10MB (Ethernet), capture size 65535
bytes
21:01:58.750072 IP 192.168.1.10.56836 > pop.someHost.gr.pop3:
Flags [S], seq 563784957, win 65535, options [mss 1460,nop,wscale
1,nop,nop,TS val 1430216395 ecr 0,sackOK,eol], length 0
E..@..@.@......
Q.h6...n!...........{.............
U?^.........
21:01:58.751523 IP 192.168.1.10.56837 > pop.someHost.gr.pop3:
Flags [S], seq 3877282998, win 65535, options [mss 1460,nop,wscale
1,nop,nop,TS val 1430216396 ecr 0,sackOK,eol], length 0
E..@K.@.@......
Q.h6...n...........{.............
U?^.........
2 packets captured
498 packets received by filter
0 packets dropped by kernel
```

To capture two packets using both the ASCII and HEX format, use *sudo tcpdump -i eth0 -c 2 -XX*. To capture 100 packets to a file named *out (-w out)* and then stop, use *sudo tcpdump –c 100 –w out*. To capture the network traffic of the entire *10.10.10.0/24* network, use *sudo tcpdump net 10.10.10.0/24*. To capture incoming traffic to *<some_host>* that is also going to port 80 (usually HTTP traffic, but you should not always trust the port number for characterising the traffic), use *sudo tcpdump dst host <some_host>* and *port 80*. You can get common port numbers and service names from */etc/services*. To capture all packet types except *ARP* and *ICMP* network packets with the more readable timestamp format, use *sudo tcpdump -ttt not arp* and not *icmp*.

To capture traffic with the HTTP port from IP address 192.168.1.1 going to 192.168.1.10, or from 192.168.1.10 to 192.168.1.1 (sample output shown):

```
$ sudo tcpdump \(src 192.168.1.1 and port 80 and dst
192.168.1.10\) or \(src 192.168.1.10 and port 80 and dst
192.168.1.1 and port 80\)
21:38:11.492218 IP 192.168.1.10.57018 > 192.168.1.1.http: Flags
[F.], seq 383, ack 72468, win 65535, length 0
21:38:11.492271 IP 192.168.1.1.http > 192.168.1.10.57017: Flags
[.], ack 385, win 6000, length 0
```

To capture traffic from the 192.168.1.0/24 network to the 10.10.10.0/24 network:

```
$ sudo tcpdump src net 192.168.1.0/24 and dst net 10.10.10.0/24
```

To check traffic of host 10.10.10.1 using UDP port 514 (usually the *syslog* server):

```
$ sudo tcpdump host 10.10.10.1 and 'udp port 514'
16:49:52.681405 IP 10.10.10.1.51787 > 10.10.10.2.syslog: SYSLOG
local7.notice, length: 156
```

To capture packets below a certain size, use *sudo tcpdump less 1024*. To capture broadcast or multicast traffic, use *sudo tcpdump 'broadcast or multicast'*. To capture and show IPv6 traffic only, use *sudo tcpdump ip6*.

## *Tcpdump* tips

When capturing network data using *tcpdump*, keep the following points in mind:

- If a parameter you are trying to use is not working as expected, check the man page. Always check the *tcpdump* man page when using it on a new system.
- When in doubt, capture everything! Remember—you can always filter later, but you cannot find a network packet that you did not capture.
- You can analyse captured data using *WireShark*.
- The main advantage of *tcpdump* is that as a command-line utility, you can use it with an SSH connection. *WireShark*, as a GUI application, has an overhead and can lose network data on a busy network—tcpdump needs less systems resources than *WireShark*.
- You can run *tcpdump* using the cron utility to capture data without being logged in at the machine.
- To capture full Ethernet frames, you should run *tcpdump* with the *–s 1514* parameters; 1514 is the maximum length of Ethernet network packets. Most of the time, you do not need the full packet. END

### Summary

You will now have become aware about the endless possibilities when capturing network data using *tcpdump*. The best way to learn is to practice and experiment. Remember that you do not have to process and analyse the captured data using *tcpdump*; other tools are more capable of doing that. I suggest you learn *WireShark*. I think that every network or Linux systems administrator should add *tcpdump* to his arsenal of tools.

### Web links and bibliography

[1] *tcpdump* and *libpcap* site: *http://www.tcpdump.org/*
[2] Internetworking with TCP/IP, Volume I, Douglas E Comer, Prentice Hall
[3] *Wireshark: http://www.wireshark.org/*

### By: Mihalis Tsoukalos

The author enjoys photography, writing articles, programming iOS devices and creating websites. You can reach him at *tsoukalos@sch.gr* or *@mactsouk*.